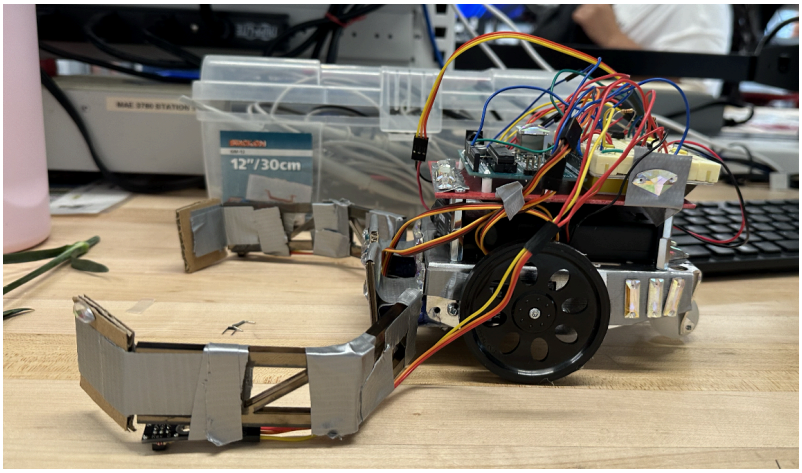
MAE 3780 Robot Project Final Report

Group 40, Usanbolor Amartuvshin (ua46), Isabel Chang (ijc35) , Helen Laird (hol3)

Robot Design and Strategy Overview

Our robot was designed around a simple cube collection mechanism consisting of two arms connected to servo motors. These servo motors allowed the arms to be deployed at the start of the match and to lift up and down during turns. The arms were initially held upwards before the match began to ensure the robot fit within the required 8x8in starting constraint, and then were moved 90° downward to fit within the 12in diameter size limit when the match began. We mounted a color sensor to the bottom of the robot and placed two QTI sensors at the end of each arm.

The workflow of the robot began with it moving forward while simultaneously moving the arms down. When the color sensor detected a color change, which indicated that the robot had reached the center of the board, the robot turned 90° to the right. During each turn, the arm on the same side as the turning direction lifted up to allow the robot to more effectively scoop and collect additional cubes. Once the turn was completed the lifted arm would lower back down. After the initial color change detection, the robot was programmed to move forward until it hit the black border, which was detected by the QTI sensors. It was also programmed to no longer respond to changes in the board's color. If the robot hit the black border, it backed away slightly and then turned 180° in the direction opposite to the arm that detected the border. This strategy allowed the robot to continue moving back and forth along the middle of the playing field to maximize cube collection. It also ensured that if the robot was bumped during play, it would not fall off the board and would still manage to collect cubes.



Design Process Reflection

Our process started with a discussion of how involved we wanted to be in the actual competition vs the milestones, understanding the rules and regulations we had to work around, and brainstorming different strategies. After a few conversations, we

decided to go with a relatively simple design that would get us through the milestones first and foremost, and hopefully hold up through competition rounds. In the end, we decided to use the color and QTI sensors with arms to gather cubes. Our group formed on the basis of prioritizing milestones over competition, which made the workflow rather simple and brainstorming was a very collaborative process.

The design presentation milestone gave us some feedback on our initial strategy from the TAs, with tips for placement of the QTIs and how to deal with the opposing robot. From here, we mainly worked on just hitting each milestone as they were due. This worked well for us as a group, as we had a deliverable each week and were all in agreement that earlier in the week was better for meeting in case any unforeseen issues arose. We ended up having one person work mainly on the code for the robot, and the other two focused more on the physical aspects as the robot gained more components. Always meeting as a full group ensured that nothing fell through the cracks and no one person was left to do all the work by themselves. Following this we were able to pass the milestones without any major issues.

When it came to getting ready for competition, we noticed the robot was falling apart and determined our previous cardboard cutouts would not hold. We exchanged the cardboard for laser-cut pieces for better durability and added glue rather than relying solely on tape. The result was more reliable and made it more competition-ready. Our final robot didn't include everything we had in the initial design presentation (like bigger wheels) as we deemed them unnecessary. We also changed the arms to help them stay attached and hold the cubes within the perimeter when turning. A large struggle was finding the correct timing for the turns to line up correctly, and to get the positional motors to rotate in order for the arms to go up and down without crashing into the table. They each ran independently to allow for one to go up while turning, depending on direction, to potentially gather more blocks, and getting that to work in the code was one of the more challenging issues we faced. On the mechanical side of things, we ran into a few issues with parts falling off, such as the arms, motors, or the rubber bands on our wheels. In the end, we loaded the robot with a mixture of hot glue, super glue, and tape to keep everything secure.

<u>Competition Analysis</u>

Overall, our robot performed better than we expected on competition day. In total, we played 7 matches, winning 3 and losing 4. Although we did not win more matches, we learned a lot about our robot's strengths and weaknesses throughout the tournament. In our first match, we won by collecting 2 cubes. We had a great opportunity to clear more of the board, but our robot kept turning much more than expected. This caused it to get stuck in a back-and-forth loop in the same area, preventing it from collecting additional cubes. Later, we realized that this happened because we had replaced the batteries with brand-new ones before the match. The

increased battery power caused the motors to run faster and made the robot overturn. Identifying this, we switched back to the older batteries, which improved the robot's control and movement. Our third match was our best performance of the day. During that match, our robot successfully collected 8 cubes. At one point, another robot nearly pushed us off the board, but our robot was still able to detect the border, turn around, and remain inside the field. This showed that our QTI's worked pretty well. During several other matches, our robot became tangled or pushed around by other robots, which often prevented us from collecting many cubes. However, one feature that worked very well was our independently moving arms. While turning, the robot could lift the arm on one side, helping it collect additional cubes more efficiently. Although this feature performed as expected, we discovered that our robot struggled to hold onto the cubes it collected. The cubes would sometimes fall out when the robot moved too quickly or when we bumped into other robots.

One unexpected advantage of our design was how the independently moving arms helped the robot escape when tangled with other robots. When the sensors detected the black border, the robot would "wiggle" while adjusting its arms independently, which sometimes allowed it to break free. In a way, this unintentionally became a defensive strategy during matches.

Overall, the greatest strength of our robot was the independently moving arms, which improved maneuverability and helped with cube collection and escaping obstacles. However, the robot's biggest weakness was not being able to securely maintain the cubes throughout the match, especially during fast movement and collisions with other robots.

Conclusions

In conclusion, if we were to complete this project again under the same constraints, we would choose a simpler design that still fulfills the competition requirements and overall goal. During the tournament, we observed many robots that used a simple square-shaped design with enclosed frames that kept the cubes secured inside the robot. Although simple, these designs performed very effectively because the outer shell added strength during matches, making the robots harder for opponents to push around while also successfully collecting and holding cubes throughout the game.

In addition, we would incorporate larger wheels and gears to increase the robot's speed. During one of the matches, two teams used nearly identical designs and strategies; however, the winning team had a faster robot, allowing them to react more quickly and collect cubes more efficiently. This demonstrated that speed can be a major factor that separates successful robots from others in the competition.

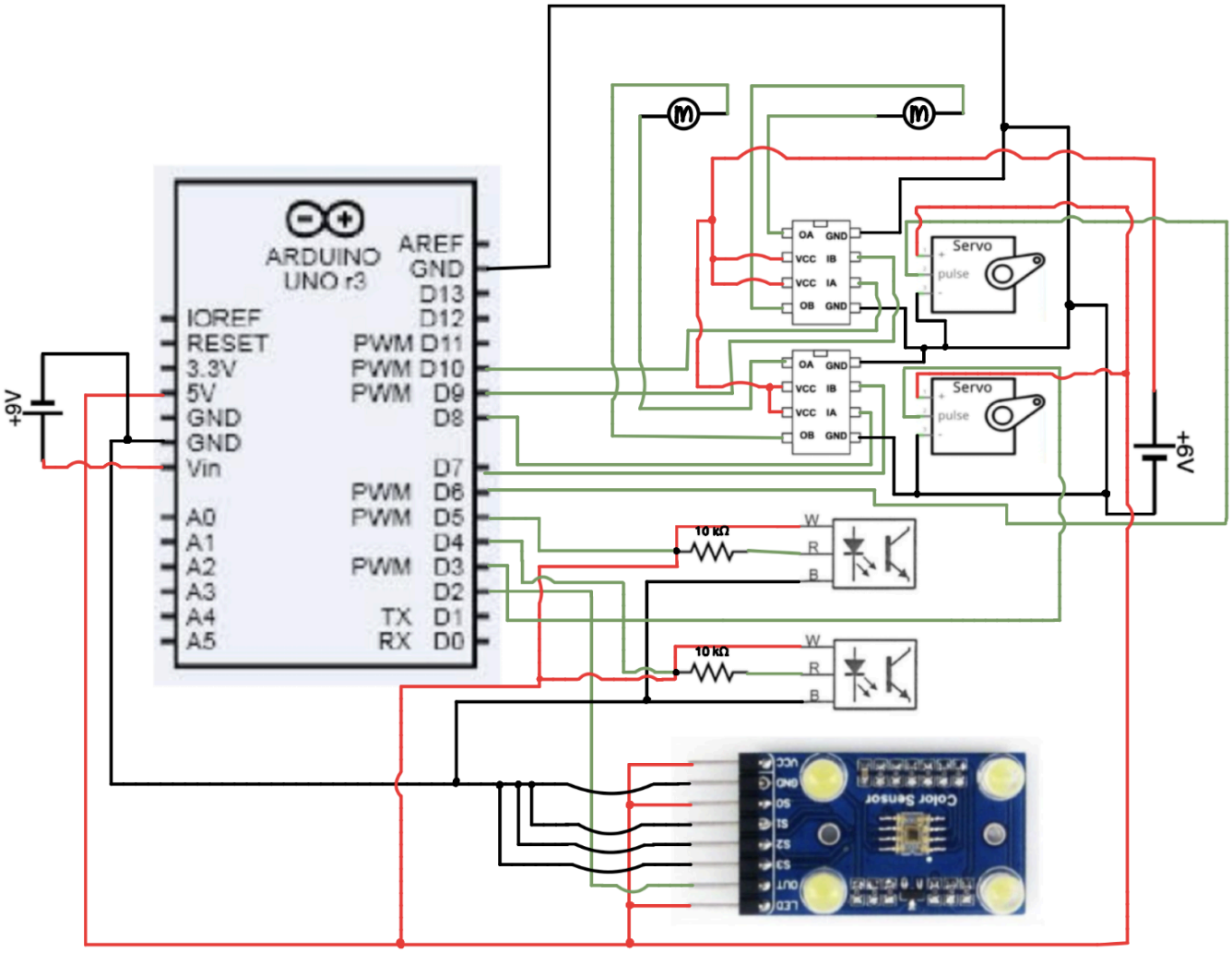
Overall, for future students participating in this project, we would recommend keeping the robot design as simple as possible while still ensuring it effectively achieves the main objective of the competition.

Appendix A: bill of materials (BOM)

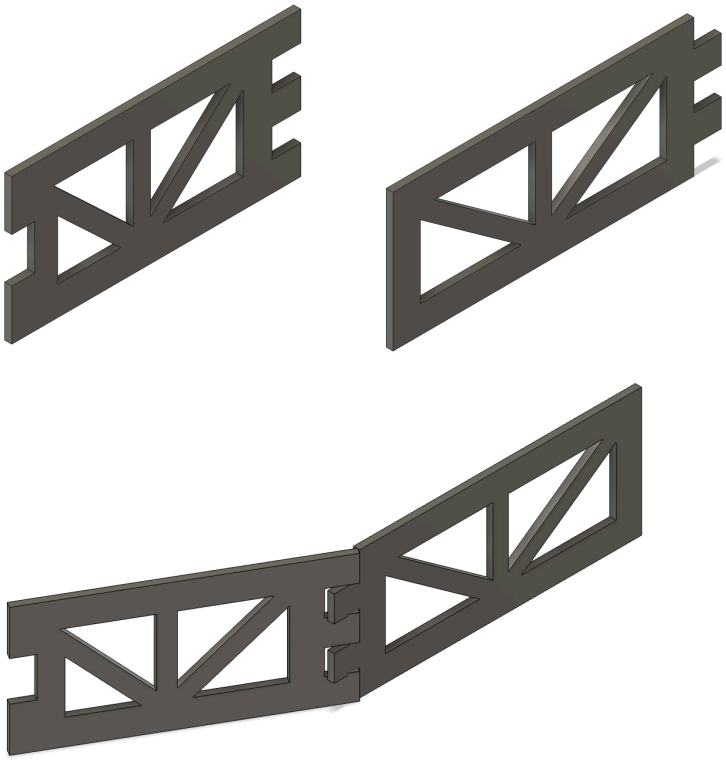
Bill of Materials

Part Name	Source	Part Description	Unit Cost ($)	Quantity	Total ($)
Positional Micro Servo	Lab	Positional micro servo	3.00	1	3.00
Duct Tape	McMaster-Carr	Roll of duct tape	7.45	1	7.45
Acrylic	RPL	12x12" acrylic sheet	5.00	1	5.00
Acrylic Cutting Charge	RPL	$0.05 per inch cut	0.05	37.39	1.86926
RPL Charge	RPL	$0.50 per part cut	0.50	4	2.00
				Total ($)	19.32
			Remaining Budget ($)		20.68

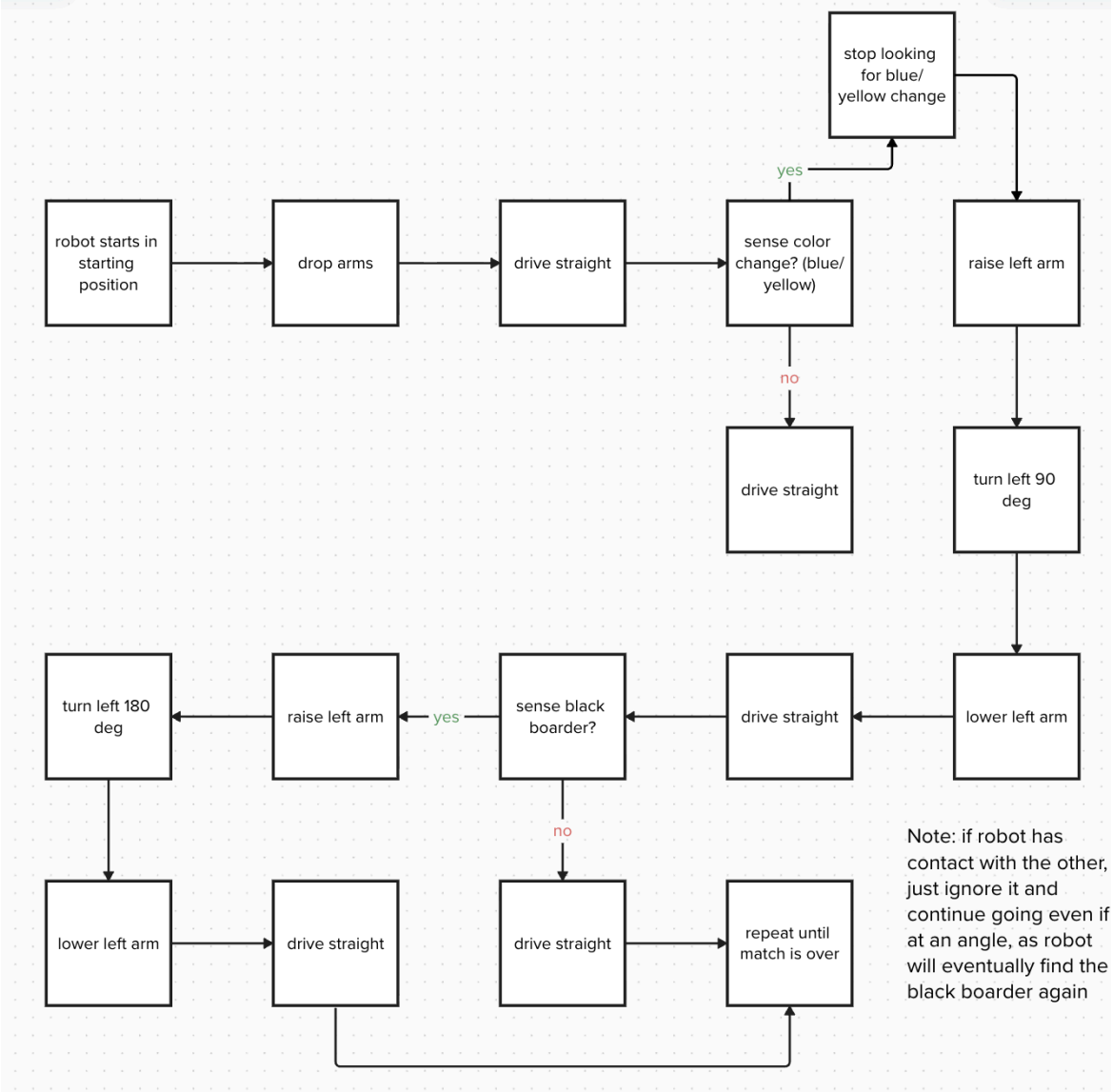
Appendix B: circuit diagram



Appendix C: CAD files and drawings



Appendix D: Flowchart



Appendix E: Code

```
#include <avr/io.h>             // AVR register
#include <avr/interrupt.h>      // interrupt functions
#include <util/delay.h>         // delay functions

#define BLUE 1                  // label blue as 1
#define YELLOW 0                // label yellow as 0
#define COLOR_THRESHOLD 250     // cutoff value for deciding blue vs yellow color

#define TURN_90_TIME 850        // right 90 turn time
#define TURN_180_TIME 1250      // right 180 turn time
#define BACK_AWAY_TIME 180      // time spent backing away from border

#define INITIAL_TURN_RIGHT 1    // 1 = turn right first, 0 = turn left first

// LEFT TURN TUNING
#define LEFT_TURN_90_TIME 700   // softer left 90 turn (motors were diff speeds)
#define LEFT_TURN_180_TIME 900  // softer left 180 turn

// ---------------- SERVO VALUES ----------------

#define LEFT_ARM_DOWN_US  1000  // left arm down pulse
#define RIGHT_ARM_DOWN_US 1900  // right arm down pulse

#define LEFT_ARM_UP_US    2000  // left arm up pulse
#define RIGHT_ARM_UP_US   1000  // right arm up pulse

volatile uint16_t leftServoUS = LEFT_ARM_DOWN_US;    // current left servo pulse
width
volatile uint16_t rightServoUS = RIGHT_ARM_DOWN_US;  // current right servo pulse
width

// ---------------- COLOR SENSOR ----------------

volatile uint16_t period_us = 0;  // stores measured pulse period
volatile uint8_t saw_rising = 0;  // tracks if rising edge happened

void initColor() {              // setup color sensor
 DDRD &= ~(1 << PD2);           // make D2 an input
```

```
 TCCR1A = 0;                    // normal timer mode
 TCCR1B = (1 << CS11);          // timer1 prescaler = 8
 TCNT1 = 0;                     // reset timer1 count

 PCICR |= (1 << PCIE2);         // enable port D interrupts
 PCMSK2 |= (1 << PCINT18);      // enable interrupt on D2
}

ISR(PCINT2_vect) {              // runs whenever D2 changes
 if (PIND & (1 << PD2)) {       // if signal went high
   TCNT1 = 0;                   // restart timer
   saw_rising = 1;              // remember rising edge happened
 } else {                       // if signal went low
   if (saw_rising) {            // only measure if rising edge happened first
     period_us = TCNT1;         // save pulse width
     saw_rising = 0;            // reset flag
   }
 }
}

uint16_t getColorPeriod() {     // get latest color period
 _delay_ms(5);                  // short wait for stable reading

 cli();                         // pause interrupts
 uint16_t p = period_us;        // copy shared variable
 sei();                         // turn interrupts back on

 return p;                      // return measured period
}

int readColor() {               // determine board color
 long sum = 0;                  // stores total of readings

 for (int i = 0; i < 5; i++) {  // take 5 readings
   sum += getColorPeriod();     // add reading to total
   _delay_ms(2);                // short wait between readings
 }

 uint16_t avg = sum / 5;        // average the readings
```

```
 if (avg > COLOR_THRESHOLD) return BLUE;  // larger value means blue
 else return YELLOW;                      // otherwise yellow
}

// ---------------- QTI ----------------

void initQTI() {                 // setup QTI sensors
 DDRD &= ~(1 << PD3);            // D3 input
 DDRD &= ~(1 << PD4);            // D4 input

 PORTD &= ~(1 << PD3);           // no pullup resistor
 PORTD &= ~(1 << PD4);           // no pullup resistor
}

int leftEdgeDetected() {         // check left sensor
 return (PIND & (1 << PD3)) ? 1 : 0;  // return 1 if border seen
}

int rightEdgeDetected() {        // check right sensor
 return (PIND & (1 << PD4)) ? 1 : 0;  // return 1 if border seen
}

// ---------------- TIMER2 / SERVOS ----------------

void initTimer2() {              // setup timer2 for servo timing
 TCCR2A = 0;                     // normal mode
 TCCR2B = (1 << CS21);           // prescaler = 8
}

void delay_us_timer2(uint16_t us) {  // microsecond delay using timer2
 while (us > 100) {              // break large delay into chunks
   TCNT2 = 0;                    // reset timer2
   while (TCNT2 < 200);          // wait 100 us
   us -= 100;                    // subtract completed delay
 }

 TCNT2 = 0;                      // reset timer2 again
 while (TCNT2 < us * 2);         // wait remaining time
}
```

```
void initServos() {               // setup servo pins
 DDRD |= (1 << PD5);              // D5 output
 DDRD |= (1 << PD6);              // D6 output
}

void servoPulseBoth() {           // send pulse to both servos
 uint16_t left = leftServoUS;     // current left pulse width
 uint16_t right = rightServoUS;   // current right pulse width

 PORTD |= (1 << PD5) | (1 << PD6);  // start both pulses high

 if (left < right) {              // if left pulse ends first
   delay_us_timer2(left);         // wait left pulse time
   PORTD &= ~(1 << PD5);          // end left pulse

   delay_us_timer2(right - left); // wait remaining right pulse time
   PORTD &= ~(1 << PD6);          // end right pulse
 } else {                         // if right pulse ends first
   delay_us_timer2(right);        // wait right pulse time
   PORTD &= ~(1 << PD6);          // end right pulse

   delay_us_timer2(left - right);      // wait remaining left pulse time
   PORTD &= ~(1 << PD5);          // end left pulse
 }

 _delay_ms(18);                   // finish servo refresh cycle
}

void delayWithServos(uint16_t ms) {  // delay while keeping servos active
 for (uint16_t i = 0; i < ms / 20; i++) {  // repeat every 20 ms
   servoPulseBoth();              // refresh servo position
 }
}

// ---------------- ARM CONTROL ----------------

void raiseLeftArm() {             // move left arm up
 leftServoUS = LEFT_ARM_UP_US;    // set left arm up
 rightServoUS = RIGHT_ARM_DOWN_US; // keep right arm down
 delayWithServos(500);            // wait for servo movement
```

```
}

void lowerLeftArm() {            // move left arm down
 leftServoUS = LEFT_ARM_DOWN_US;           // set left arm down
 rightServoUS = RIGHT_ARM_DOWN_US;      // keep right arm down
 delayWithServos(500);           // wait for servo movement
}

void raiseRightArm() {           // move right arm up
 leftServoUS = LEFT_ARM_DOWN_US;  // keep left arm down
 rightServoUS = RIGHT_ARM_UP_US;  // set right arm up
 delayWithServos(500);           // wait for servo movement
}

void lowerRightArm() {           // move right arm down
 leftServoUS = LEFT_ARM_DOWN_US;  // keep left arm down
 rightServoUS = RIGHT_ARM_DOWN_US;      // set right arm down
 delayWithServos(500);           // wait for servo movement
}

// ---------------- MOTORS ----------------

void initMotors() {              // set up motor pins
 DDRD |= (1 << PD7);             // D7 output
 DDRB |= (1 << PB0);             // D8 output
 DDRB |= (1 << PB1);             // D9 output
 DDRB |= (1 << PB2);             // D10 output
}

void setMotors(uint8_t pattern) { // control motors with bit pattern
 if (pattern & 0b0001) PORTD |= (1 << PD7);  // set D7 high
 else PORTD &= ~(1 << PD7);                  // set D7 low

 if (pattern & 0b0010) PORTB |= (1 << PB0);  // set D8 high
 else PORTB &= ~(1 << PB0);                  // set D8 low

 if (pattern & 0b0100) PORTB |= (1 << PB1);  // set D9 high
 else PORTB &= ~(1 << PB1);                  // set D9 low

 if (pattern & 0b1000) PORTB |= (1 << PB2);  // set D10 high
```

```
 else PORTB &= ~(1 << PB2);                // set D10 low
}

void stopNow() {                 // stop robot
 setMotors(0b0000);               // turn all motors off
 servoPulseBoth();                // keep servos refreshed
}

void driveForwardStep() {         // drive forward briefly
 setMotors(0b1001);               // forward motor pattern
 servoPulseBoth();                // refresh servos
}

void backAway() {                // reverse away from border
 setMotors(0b0110);               // backward motor pattern
 delayWithServos(BACK_AWAY_TIME);           // keep reversing
 stopNow();                      // stop after backing up
}

// ---------------- SOFTER LEFT TURN ----------------

void softLeftTurn(uint16_t totalTime) {     // smoother left turn
 uint16_t elapsed = 0;            // tracks total turn time

 while (elapsed < totalTime) {        // keep turning until done
   setMotors(0b1010);             // left turn motor pattern
   delayWithServos(40);           // motors on
   elapsed += 40;                // add elapsed time

   stopNow();                    // short coast
   delayWithServos(20);           // motors off briefly
   elapsed += 20;                // add elapsed time
 }

 stopNow();                      // fully stop after turn
}

// ---------------- TURNING ----------------

void turnRight90() {             // right 90 turn
```

```
 raiseLeftArm();                // lift left arm
 setMotors(0b0101);             // right turn motor pattern
 delayWithServos(TURN_90_TIME);      // keep turning
 stopNow();                     // stop motors
 lowerLeftArm();                // lower left arm
}

void turnLeft90() {             // left 90 turn
 raiseRightArm();               // lift right arm
 softLeftTurn(LEFT_TURN_90_TIME);    // softer left turn
 lowerRightArm();               // lower right arm
}

void turnRight180() {           // right 180 turn
 raiseLeftArm();                // lift left arm
 setMotors(0b0101);             // right turn motor pattern
 delayWithServos(TURN_180_TIME);     // keep turning
 stopNow();                     // stop motors
 lowerLeftArm();                // lower left arm
}

void turnLeft180() {            // left 180 turn
 raiseRightArm();               // lift right arm
 softLeftTurn(LEFT_TURN_180_TIME);  // softer left turn
 lowerRightArm();               // lower right arm
}

// ---------------- MOVE + DEPLOY ARMS ----------------

void driveForwardDeployArms(uint8_t cycles) {   // move forward while lowering arms
 leftServoUS = LEFT_ARM_DOWN_US;          // left arm down
 rightServoUS = RIGHT_ARM_DOWN_US;       // right arm down

 setMotors(0b1001);             // start driving forward

 for (uint8_t i = 0; i < cycles; i++) {          // repeat servo refreshes
   servoPulseBoth();            // keep arms moving/holding
 }
}
```

```
// ---------------- MAIN ----------------

int main(void) {                // main program starts here
 //Serial.begin(9600);           // serial debugging if needed

 initMotors();                  // setup motors
 initColor();                   // setup color sensor
 initQTI();                     // setup QTI sensors
 initTimer2();                  // setup timer2
 initServos();                  // setup servos

 sei();                         // enable interrupts

 driveForwardDeployArms(75);      // move forward while dropping arms

 int startColor = readColor();    // save starting color

 while (1) {                    // keep driving until color changes
   int currentColor = readColor();   // read the current color

   if (currentColor != startColor) {   // if different color found
     stopNow();                  // stop robot
     break;                     // leave loop
   }

   driveForwardStep();           // continue driving
 }

 delayWithServos(200);           // short pause

#if INITIAL_TURN_RIGHT           // choose initial turn direction
 turnRight90();                 // turn right first
#else
 turnLeft90();                  // turn left first
#endif

 delayWithServos(200);           // short pause after turning

 while (1) {                    // main driving loop
   leftServoUS = LEFT_ARM_DOWN_US;    // keep left arm down
```

```
  rightServoUS = RIGHT_ARM_DOWN_US;     // keep right arm down

  if (leftEdgeDetected()) {      // if left sensor sees border
    stopNow();                  // stop robot
    backAway();                  // reverse away
    turnRight180();              // turn away from border
  }
  else if (rightEdgeDetected()) {    // if right sensor sees border
    stopNow();                  // stop robot
    backAway();                  // reverse away
    turnLeft180();              // turn away from border
  }

  driveForwardStep();           // keep moving forward
 }
}
```